ЛАБОРАТОРНАЯ РАБОТА № 7

Создание графовой модели Neo4j и запросов Cypher

Дисциплина: «Проектирование и применение NoSQL-технологий»
Образовательная программа: «Информационные системы» · 3 курс
Продолжительность: 2 академических часа · Форма: индивидуальная
Связанная лекция № 7: «Графовая модель, обходы, Cypher и варианты применения»

1. Цель работы

Сформировать практические навыки анализа предметной области, проектирования property graph-модели, создания узлов и направленных отношений в Neo4j и реализации запросов Cypher для поиска связанных данных и многошаговых обходов.

2. Результаты обучения

После выполнения работы студент анализирует паттерны доступа; проектирует узлы, метки, свойства и типы отношений; создаёт граф в Neo4j; применяет CREATE/MERGE и MATCH; выполняет фильтрацию и многошаговые обходы; изменяет и удаляет элементы графа; оценивает преимущества и ограничения графовой модели; обосновывает архитектурное решение.

3. Необходимые знания

Перед занятием повторить: NoSQL-модели; сущности и связи; граф, вершина и ребро; направленные связи; идентификаторы; паттерны доступа; основы Cypher из лекции № 7.

4. Программное обеспечение

Neo4j Desktop или Neo4j AuraDB Free; Neo4j Browser/Query; альтернативно Docker. Для учебной работы достаточно 4 ГБ ОЗУ и современного браузера. Используйте актуальную стабильную версию Neo4j, доступную в выбранной среде.

5. Безопасность

- Использовать только искусственные учебные данные, без реальных персональных данных.
- Не публиковать пароли, URI и секреты AuraDB.
- Не открывать Bolt/HTTP-порты во внешнюю сеть без необходимости.

- Перед DELETE/DETACH DELETE сначала выполнить MATCH ... RETURN и проверить выбранные элементы.
- Не выполнять команды очистки на рабочих базах данных.

6. Краткая теория

6.1. Property graph

В Neo4j данные удобно представлять как граф свойств: узлы (nodes) описывают объекты, отношения (relationships) соединяют узлы, а свойства (properties) хранят атрибуты. Метки (labels) группируют узлы по ролям, например Student, Course или Teacher. Связь имеет тип и направление, например (:Student)-[:ENROLLED_IN]->(:Course).

6.2. Когда графовая модель оправдана

Графовая модель особенно полезна, когда ключевые запросы зависят не только от атрибутов объектов, но и от цепочек связей: друзья друзей, пререквизиты дисциплины, маршруты, рекомендации, зависимости компонентов, связи клиентов и операций. Если основная задача — хранение автономных документов без интенсивных обходов, документная модель может быть проще.

6.3. Cypher

Cypher — декларативный язык запросов к property graph. Шаблон графа записывается визуально: узел — в круглых скобках, отношение — в квадратных, направление — стрелкой.

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course)
RETURN s.name, c.title;
```

6.4. CREATE и MERGE

CREATE создаёт новый элемент. MERGE ищет заданный шаблон и создаёт его, если совпадение отсутствует. MERGE удобен при повторяемой загрузке, но шаблон и уникальные идентификаторы должны быть спроектированы осознанно.

```
CREATE (:Student {studentId: 1001, name: 'Aruzhan'});
MERGE (c:Course {code: 'NOSQL301'})
SET c.title = 'NoSQL Technologies';
```

6.5. Обходы

Главное преимущество графовой модели проявляется в обходах. Запрос может искать непосредственную связь или путь длиной в несколько отношений. Глубину следует ограничивать, поскольку неограниченные обходы на больших графах могут быть дорогими.

```
MATCH (c:Course {code:'ADV401'})-[:REQUIRES*1..3]->(p:Course)
RETURN DISTINCT p.code, p.title;
```

7. Постановка задачи

Сквозной пример — информационная система университета. Необходимо представить студентов, преподавателей и дисциплины как граф, чтобы отвечать на вопросы о регистрациях, преподавании и пререквизитах. Студент должен начать не с таблиц, а с пользовательских вопросов: какие связанные объекты нужно находить и по каким путям.

Элемент	Пример	Назначение
Узел	Student	студент
Узел	Course	дисциплина
Узел	Teacher	преподаватель
Связь	ENROLLED_IN	регистрация на курс
Связь	TEACHES	преподавание
Связь	REQUIRES	пререквизит

8. Подготовка среды

Вариант A — Neo4j Desktop/AuraDB: создайте учебную БД и откройте Query/Browser. Вариант B — локальный Docker-контейнер:

```
docker run --name neo4j-lab7 -p 127.0.0.1:7474:7474 -p 127.0.0.1:7687:7687 -e
NEO4J_AUTH=neo4j/Lab7StrongPass123 -d neo4j:latest
```

Откройте http://localhost:7474, войдите с указанными учебными учётными данными. Не используйте этот пароль вне лабораторной среды. Проверка соединения:

```
RETURN 'Neo4j works' AS status;
```

9. Демонстрационный пример

Создадим минимальный граф университета. Пример демонстрирует синтаксис, но не решает индивидуальный вариант.

```
MERGE (s1:Student {studentId:1001}) SET s1.name='Aruzhan'
MERGE (s2:Student {studentId:1002}) SET s2.name='Dias'
MERGE (c1:Course {code:'NOSQL301'}) SET c1.title='NoSQL Technologies'
MERGE (c2:Course {code:'DB201'}) SET c2.title='Database Systems'
MERGE (t:Teacher {teacherId:501}) SET t.name='A. Teacher'
MERGE (s1)-[:ENROLLED_IN {semester:'2026-Fall'}]->(c1)
MERGE (s2)-[:ENROLLED_IN {semester:'2026-Fall'}]->(c1)
MERGE (t)-[:TEACHES]->(c1)
MERGE (c1)-[:REQUIRES]->(c2);
```

Проверка структуры:

```
MATCH (n) RETURN labels(n) AS labels, count(*) AS qty;
MATCH ()-[r]->() RETURN type(r) AS relation, count(*) AS qty;
```

Получить студентов курса:

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {code:'NOSQL301'})
RETURN s.studentId, s.name ORDER BY s.name;
```

Изменение свойства и безопасное удаление тестовой связи:

```
MATCH (s:Student {studentId:1001}) SET s.name='Aruzhan S.' RETURN s;
MATCH (s:Student {studentId:1002})-[r:ENROLLED_IN]->(c:Course {code:'NOSQL301'}) RETURN
s,r,c;
MATCH (s:Student {studentId:1002})-[r:ENROLLED_IN]->(c:Course {code:'NOSQL301'}) DELETE
r;
```

10. Пошаговый порядок выполнения

Этап 1. Анализ требований

Определите пользователей, 5–7 ключевых вопросов к данным, операции чтения/записи и те вопросы, где требуется переход по связям.

Этап 2. Выбор модели

Сравните графовую модель минимум с документной. Объясните, почему связи и обходы существенны для вашего варианта.

Этап 3. Проектирование

Представьте схему: минимум 3 типа узлов, 3 типа отношений, обязательные свойства, уникальные идентификаторы, направление отношений и пример пути.

Этап 4. Создание графа

Создайте не менее 15 узлов и 20 отношений. Используйте осмысленные учебные данные.

Этап 5. Запросы

Реализуйте минимум 7 запросов: простое MATCH, фильтрацию, сортировку/ограничение, запрос по отношению, двухшаговый обход, обход переменной длины с ограниченной глубиной, агрегирование count().

Этап 6. Изменение

Измените минимум одно свойство узла или отношения и подтвердите результат запросом.

Этап 7. Удаление

Сначала покажите выбранный объект/отношение через RETURN, затем безопасно удалите одну тестовую связь или узел.

Этап 8. Анализ

Опишите эффективные и сложные запросы, возможные избыточные связи, ограничения модели и сценарий роста графа.

11. Основное задание

1. Составить таблицу 5–7 паттернов доступа.
2. Спроектировать property graph с минимум 3 метками узлов и 3 типами отношений.
3. Создать минимум 15 узлов и 20 отношений.
4. Реализовать минимум 7 запросов Cypher, включая двухшаговый и переменной длины обход.
5. Выполнить изменение и безопасное удаление.
6. Выполнить одно агрегирующее задание.
7. Сравнить графовую модель с документной для своего кейса.
8. Сформулировать преимущества, ограничения и критерии масштабирования.

12. Индивидуальные варианты

№	Предметная область	Узлы	Отношения	Повышенное задание
1	Университет	Student, Course, Teacher	ENROLLED_IN, TEACHES, REQUIRES	Найти цепочку пререквизитов до 3 уровней
2	Социальная сеть	User, Post, Topic	FOLLOWS, CREATED, INTERESTED_IN	Найти рекомендации через связи друзей/интересов
3	Интернет-магазин	Customer, Product, Category	BOUGHT, VIEWED, BELONGS_TO	Найти товары, связанные через покупки пользователей
4	Библиотека	Reader, Book, Author	BORROWED, WROTE, INTERESTED_IN	Найти книги через общих авторов/интересы
5	Служба доставки	Courier, Zone, Order	SERVES, DELIVERS, NEXT_TO	Найти связанные зоны до 2–3 переходов
6	Система бронирования	User, Resource, Location	BOOKED, LOCATED_IN, NEAR	Найти альтернативные связанные ресурсы
7	Медицинская учебная система	Doctor, Department, Specialization	WORKS_IN, HAS_SPECIALIZATION, COOPERATES_WITH	Найти специалистов через подразделения; только синтетические данные
8	Склад и	Warehouse,	STORES,	Найти

	поставки	Product, Supplier	SUPPLIES, CONNECTED_TO	альтернативных поставщиков и связанные склады
9	Онлайн-обучение	Student, Course, Skill	ENROLLED_IN, DEVELOPS, REQUIRES_SKILL	Найти учебную траекторию через навыки
10	ИТ-инфраструктура	Service, Server, Database	RUNS_ON, DEPENDS_ON, CONNECTS_TO	Найти зависимости сервиса до 3 уровней

Для каждого варианта обязательны: минимум 7 запросов; одно изменение; одно безопасное удаление; count()-аналитика; один многошаговый обход; обоснование графовой модели. Варианты сопоставимы по сложности.

13. Задания по уровням сложности

Уровень	Обязательные результаты	Максимальная оценка
Базовый	Схема, 15 узлов, 20 отношений, CREATE/MERGE, MATCH, фильтрация, изменение, удаление	до 69
Средний	Все базовое + 7 запросов, агрегирование, 2-шаговый обход, ограничения/идентификаторы	70–89
Повышенный	Все среднее + ограниченный variable-length path, сравнение моделей, анализ производительности/масштабирования	90–100

14. Ожидаемые результаты и проверка

- Все типы узлов и отношений соответствуют схеме.
- Запросы возвращают ожидаемые связанные объекты, а не случайный декартов продукт.
- Направления отношений отражают семантику предметной области.
- Обход переменной длины имеет разумную границу глубины.
- После изменения контрольный MATCH показывает новое значение.
- Перед удалением показан объект/отношение, после удаления запрос подтверждает отсутствие выбранного элемента.

15. Типичные ошибки

Ошибка	Последствие	Исправление
Перенос реляционной схемы 1:1	Граф не отражает реальные паттерны обхода	Начать с вопросов и путей
Связь хранится строковым ID вместо relationship	Нельзя естественно обходить граф	Моделировать значимую связь ребром
Неверное направление	Запросы становятся неясными	Выбрать семантически читаемое направление
Дубли узлов из-за CREATE	Искажённые результаты	Уникальные ID и осознанный MERGE
Свойство превращено в отдельный узел без причины	Избыточная сложность	Узел создавать, когда объект участвует в связях
Неограниченный *-обход	Риск дорогого запроса	Ограничивать глубину
DELETE без предварительной проверки	Удаление неверных данных	Сначала MATCH ... RETURN
Реальные персональные данные	Риск нарушения приватности	Только синтетические данные

16. Контрольные вопросы

9. Что такое property graph?
10. Чем узел отличается от отношения и свойства?
11. Для чего используются labels в Neo4j?
12. Чем CREATE отличается от MERGE?
13. Как MATCH описывает графовый шаблон?
14. Почему направление отношения важно?
15. Когда атрибут следует моделировать свойством, а когда отдельным узлом?
16. Что показывает двухшаговый обход?
17. Почему глубину variable-length path следует ограничивать?
18. Почему графовая модель может быть лучше документной для задач анализа связей?

17. Требования к отчёту

19. Титульный лист.
20. Номер и точная тема работы.
21. Цель и индивидуальный вариант.
22. Описание предметной области и пользователей.
23. Таблица паттернов доступа.
24. Обоснование Neo4j и сравнение с документной моделью.
25. Схема графа: узлы, labels, свойства, отношения и направления.
26. Cypher-команды создания данных.
27. Не менее 7 запросов с пояснениями.

28. Скриншоты результатов с подписями.
29. Изменение и безопасное удаление.
30. Анализ преимуществ, ограничений и масштабирования.
31. Ответы на контрольные вопросы.
32. Вывод.

18. Критерии оценивания

Критерий	Баллы
Анализ требований	10
Обоснование графовой модели	10
Проектирование структуры графа	15
Корректность тестовых данных	10
Основные операции Cypher	20
Обходы и сложные запросы	15
Анализ результатов	10
Оформление отчёта	5
Защита	5
Итого	100

90–100 — отлично; 70–89 — хорошо; 50–69 — удовлетворительно; 0–49 — требуется повторное выполнение. Полное выполнение означает корректный и воспроизводимый результат с обоснованием; частичное — отдельные пропуски или ошибки; невыполнение — отсутствие результата либо невозможность объяснить модель и запросы.

19. Вопросы для защиты

33. Почему для вашего кейса выбрана графовая модель?
34. Какие узлы и отношения являются центральными?
35. Почему отношение направлено именно так?
36. Какой запрос является главным паттерном доступа?
37. Что делает ваш многошаговый обход?
38. Где возможны дубли узлов и как их избежать?
39. Какие ограничения/уникальные идентификаторы нужны?
40. Какой запрос станет дорогим при росте графа?
41. Когда документная модель была бы проще?
42. Как бы вы масштабировали или разделили граф при существенном росте системы?

20. Необязательное задание повышенной сложности

Выберите один ключевой обход и сравните два варианта моделирования одной связи: (A) relationship в Neo4j; (B) хранение списка идентификаторов в документе. Для каждого подхода оцените удобство чтения, обновления, многошагового поиска и риск рассинхронизации.

Затем реализуйте в Neo4j ограниченный путь длиной 1..3 и объясните результат. Дополнительное качество учитывается внутри существующих критериев; итог не превышает 100 баллов.

21. Источники

43. Силлабус дисциплины «Проектирование и применение NoSQL-технологий», 2026–2027: лекция № 7 и лабораторная работа № 7.
44. Neo4j. Cypher Manual — официальная документация языка Cypher.
45. Neo4j. Graph database concepts / property graph model — официальная документация.
46. Kleppmann, M. Designing Data-Intensive Applications. O’Reilly Media, 2017.

22. Итог

Ключевой результат работы — не набор команд Cypher, а обоснованная графовая модель, построенная от пользовательских вопросов. Студент должен уметь объяснить, почему конкретный объект является узлом, почему связь моделируется relationship, какие пути выполняются часто и почему Neo4j подходит для этих паттернов доступа.